

Структура DAG

В ЭТОМ ВИДЕО

- ✓ Напишете простой DAG с использованием:
 - Operator
 - Sensor
 - Hook
- ✓ Установите зависимости между Task
- ✓ Запустите DAG через Webserver
- ✓ Проанализируете весь процесс пошагово

Основные сущности Airflow

- DAG (Directed Acyclic Graph)
- Task
- Operator
- Sensor
- Hook
- XCom

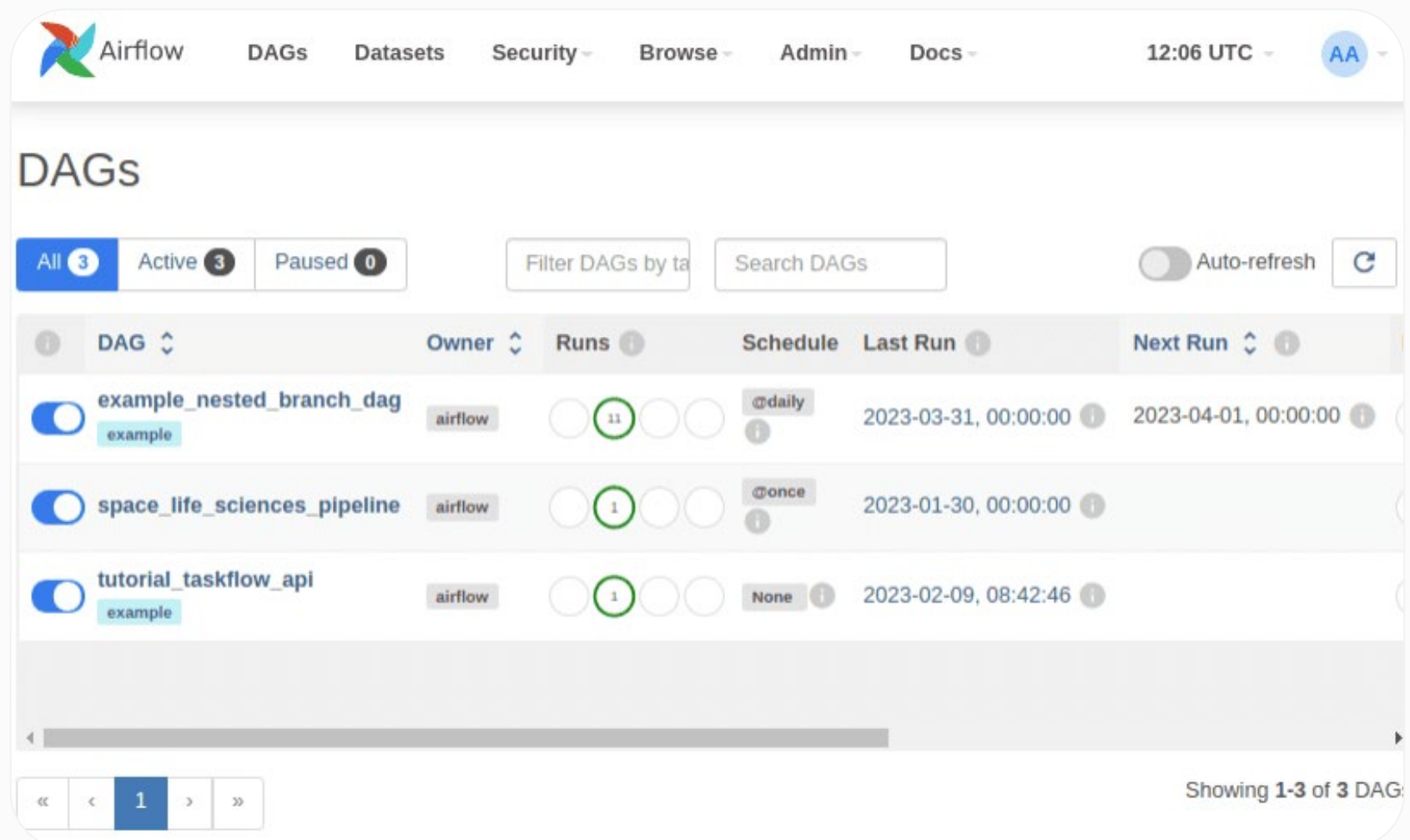
Структура DAG

DAG

DAG (Directed Acyclic Graph) — смысловое объединение задач, которые необходимо выполнить в строго определённой последовательности согласно указанному расписанию.

DAG

При помощи DAG в Airflow описываются процессы обработки данных или пайплайны.



The screenshot shows the Apache Airflow web interface. At the top, there's a navigation bar with the Airflow logo and links to DAGs, Datasets, Security, Browse, Admin, and Docs. The current time is 12:06 UTC. Below the navigation bar, the 'DAGs' section is active. It includes filters for 'All 3', 'Active 3', and 'Paused 0'. There are also input fields for 'Filter DAGs by tag' and 'Search DAGs', and a toggle for 'Auto-refresh'. The main content is a table listing DAGs. The table has columns for DAG name, Owner, Runs, Schedule, Last Run, and Next Run. Three DAGs are listed: 'example_nested_branch_dag' (owner: airflow, runs: 11, schedule: @daily, last run: 2023-03-31, 00:00:00, next run: 2023-04-01, 00:00:00), 'space_life_sciences_pipeline' (owner: airflow, runs: 1, schedule: @once, last run: 2023-01-30, 00:00:00, next run: none), and 'tutorial_taskflow_api' (owner: airflow, runs: 1, schedule: None, last run: 2023-02-09, 08:42:46, next run: none). At the bottom, there's a pagination bar showing 'Showing 1-3 of 3 DAGs'.

DAG	Owner	Runs	Schedule	Last Run	Next Run
☒ example_nested_branch_dag example	airflow	11	@daily	2023-03-31, 00:00:00	2023-04-01, 00:00:00
☒ space_life_sciences_pipeline	airflow	1	@once	2023-01-30, 00:00:00	
☒ tutorial_taskflow_api example	airflow	1	None	2023-02-09, 08:42:46	

DAG

```
with DAG(
    dag_id="example_bash_operator",
    schedule="0 0 * * *",
    start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),
    catchup=False,
    dagrun_timeout=datetime.timedelta(minutes=60),
    tags=["example", "example2"],
    params={"example_key": "example_value"},
) as dag:
    run_this_last = EmptyOperator(
        task_id="run_this_last",
    )

    # [START howto_operator_bash]
    run_this = BashOperator(
        task_id="run_after_loop",
        bash_command="echo 1",
    )

    run_this >> run_this_last
```

DAG

```
with DAG(
    dag_id="example_bash_operator",
    schedule="0 0 * * *",
    start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),
    catchup=False,
    dagrun_timeout=datetime.timedelta(minutes=60),
    tags=["example", "example2"],
    params={"example_key": "example_value"},
) as dag:
    run_this_last = EmptyOperator(
        task_id="run_this_last",
    )

    # [START howto_operator_bash]
    run_this = BashOperator(
        task_id="run_after_loop",
        bash_command="echo 1",
    )

    run_this >> run_this_last
```

DAG

```
with DAG(  
    dag_id="example_bash_operator",  
    schedule="0 0 * * *",  
    start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),  
    catchup=False,  
    dagrun_timeout=datetime.timedelta(minutes=60),  
    tags=["example", "example2"],  
    params={"example_key": "example_value"},  
) as dag:
```

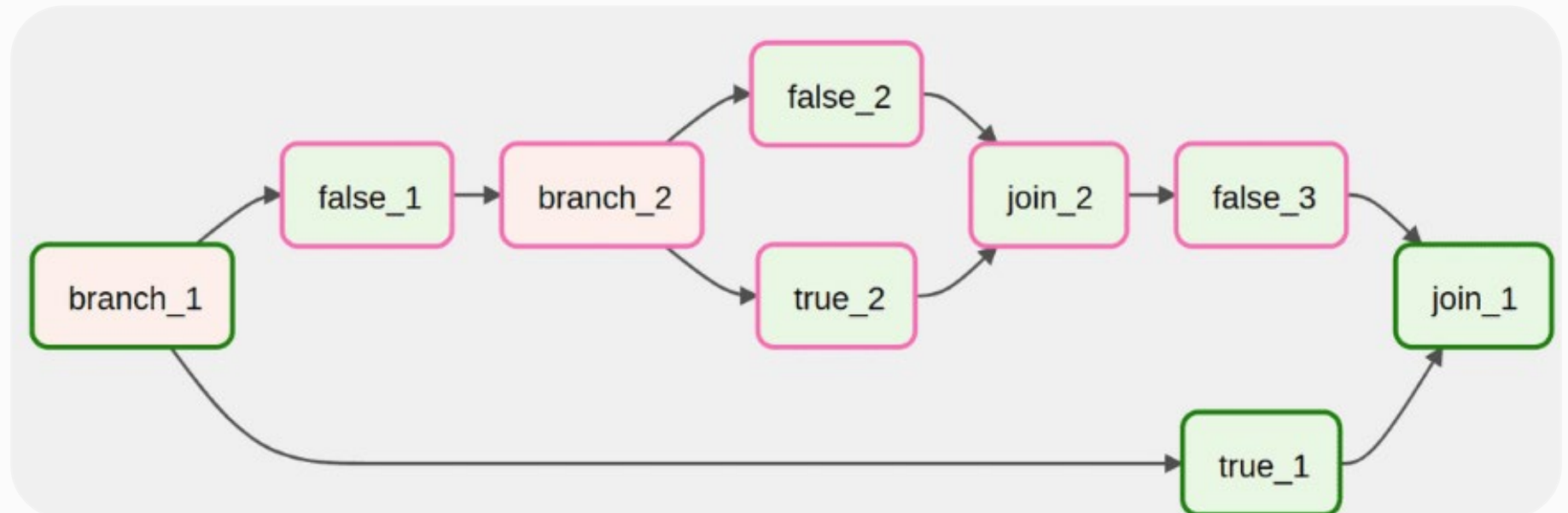


Task

Задачи (Task) — это операции, применяемые к данным, например: загрузка данных из различных источников, их агрегирование, индексирование, очистка от дубликатов, сохранение полученных результатов и прочие ETL-процессы.

На уровне кода задачи могут представлять собой Python-функции или Bash-скрипты.

Task



Task

```
with DAG(
    dag_id="example_bash_operator",
    schedule="0 0 * * *",
    start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),
    catchup=False,
    dagrun_timeout=datetime.timedelta(minutes=60),
    tags=["example", "example2"],
    params={"example_key": "example_value"},
) as dag:
    run_this_last = EmptyOperator(
        task_id="run_this_last",
    )

    run_this = BashOperator(
        task_id="run_after_loop",
        bash_command="echo 1",
    )

    run_this >> run_this_last
```

Task

```
with DAG(
    dag_id="example_bash_operator",
    schedule="0 0 * * *",
    start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),
    catchup=False,
    dagrun_timeout=datetime.timedelta(minutes=60),
    tags=["example", "example2"],
    params={"example_key": "example_value"},
) as dag:
    run_this_last = EmptyOperator(
        task_id="run_this_last",
    )

    run_this = BashOperator(
        task_id="run_after_loop",
        bash_command="echo 1",
    )

run_this >> run_this_last
```

Operator

Операторы (Operator) чаще всего отвечают за реализацию задач, по сути — это шаблон для выполнения задач.

Задачи описывают, какие действия выполнять с данными.

Операторы описывают, как эти действия выполнять.

Operator

```
with DAG(
    dag_id="example_bash_operator",
    schedule="0 0 * * *",
    start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),
    catchup=False,
    dagrun_timeout=datetime.timedelta(minutes=60),
    tags=["example", "example2"],
    params={"example_key": "example_value"},
) as dag:
    run_this_last = EmptyOperator(
        task_id="run_this_last",
    )

    run_this = BashOperator(
        task_id="run_after_loop",
        bash_command="echo 1",
    )

    run_this >> run_this_last
```

Sensor

Сенсоры (Sensor) — особая группа операторов, которые позволяют прописывать реакцию на определённое событие.

В качестве триггера может выступать наступление конкретного времени, получение некоторого файла или строки с данными, другой DAG/Task и так далее.

Sensor

```
with DAG(
    dag_id="example_sensors",
    schedule=None,
    start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),
    catchup=False,
    tags=["example"],
) as dag:
    t6 = FileSensor(task_id="wait_for_file",
filepath="/tmp/temporary_file_for_testing")

    t7 = BashOperator(
        task_id="create_file_after_3_seconds",
bash_command="sleep 3; touch /tmp/..."
    )

    ....

    tx.trigger_rule = TriggerRule.NONE_FAILED
    [t0, t0a, t1, t1a, t2, t2a, t3, t4] >> tx
    t5 >> t6 >> tx
```


Sensor

```
with DAG(
    dag_id="example_sensors",
    schedule=None,
    start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),
    catchup=False,
    tags=["example"],
) as dag:
    t6 = FileSensor(task_id="wait_for_file",
filepath="/tmp/temporary_file_for_testing")

    t7 = BashOperator(
        task_id="create_file_after_3_seconds",
bash_command="sleep 3; touch /tmp/..."
    )

    ....

tx.trigger_rule = TriggerRule.NONE_FAILED
[t0, t0a, t1, t1a, t2, t2a, t3, t4] >> tx
t5 >> t6 >> tx
```

Hooks

Hooks (Хуки) — это внешние интерфейсы для работы с различными сервисами: базы данных, внешние API-ресурсы, распределённые хранилища и так далее.

Hooks являются строительными блоками операторов и берут на себя всю логику по взаимодействию с хранилищем конфигов и доступов.

Hooks

```
def perform_sql_query():
    hook = BaseHook.get_hook('your_connection_id')
    conn = hook.get_conn()
    cursor = conn.cursor()
    query = "SELECT * FROM your_table"
    cursor.execute(query)
    results = cursor.fetchall()
    cursor.close()
    conn.close()
    return results

dag = DAG('example_sql_dag',
          default_args=default_args,
          description='An example DAG with SQL Hook',
          schedule_interval=timedelta(days=1),
          )

sql_query_task = PythonOperator(
    task_id='sql_query_task',
    python_callable=perform_sql_query,
    dag=dag,
)
```

Hooks

```
def perform_sql_query():
    hook = BaseHook.get_hook('your_connection_id')
    conn = hook.get_conn()
    cursor = conn.cursor()
    query = "SELECT * FROM your_table"
    cursor.execute(query)
    results = cursor.fetchall()
    cursor.close()
    conn.close()
    return results

dag = DAG('example_sql_dag',
          default_args=default_args,
          description='An example DAG with SQL Hook',
          schedule_interval=timedelta(days=1),
          )

sql_query_task = PythonOperator(
    task_id='sql_query_task',
    python_callable=perform_sql_query,
    dag=dag,
)
```

XCom

XCom (кросс-коммуникация) — таблица, в которой хранятся пары ключ-значение, а также отслеживаются, какая пара была предоставлена какой задачей и DAG.

XCom обеспечивает способ обмена сообщениями или данными между различными задачами или между операторами.

XCom

```
def process_data(**kwargs):
    ti = kwargs['ti']
    data = ti.xcom_pull(task_ids='sql_query_task')
    processed_data = data
    return processed_data

dag = DAG('example_sql_dag',
          default_args=default_args,
          description='An example DAG with SQL Hook',
          schedule_interval=timedelta(days=1),
          )

process_data_task = PythonOperator(
    task_id='process_data_task',
    python_callable=process_data,
    provide_context=True,
    dag=dag,
)
```

XCom

```
def process_data(**kwargs):  
    ti = kwargs['ti']  
    data = ti.xcom_pull(task_ids='sql_query_task')  
    processed_data = data  
    return processed_data  
  
dag = DAG('example_sql_dag',  
          default_args=default_args,  
          description='An example DAG with SQL Hook',  
          schedule_interval=timedelta(days=1),  
          )  
  
process_data_task = PythonOperator(  
    task_id='process_data_task',  
    python_callable=process_data,  
    provide_context=True,  
    dag=dag,  
    )
```

Преимущества Airflow

- Открытый исходный код на основе Python
- Богатый инструментарий и дружелюбный UI
- Интеграция со множеством источников данных и сервисов
- Кастомизация
- Масштабируемость
- Мониторинг и алертинг
- Возможность настройки ролевого доступа
- Поддержка тестирования

Недостатки Airflow

- При проектировании задач важно соблюдать идемпотентность
- Необходимо разобраться в механизмах обработки `execution_date`
- Нет возможности спроектировать DAG в графическом виде, как это, например, доступно в Apache NiFi
- Незначительные временные задержки в запуске задач из-за нюансов работы планировщика, связанных с расходами на постановку задач в очередь